GUÍA DEL PROYECTO

ChatBot Cognitivo

Arquitectura, conversación y análisis cualitativo

Una explicación del funcionamiento real del código, sus carpetas, la base de datos, los indicadores, los reportes y la exportación a Excel. Pensada para comprender, mantener y presentar el sistema.

Versión documentada	Alcance
29 de octubre de 2026	Cambios preparados en el proyecto local. La publicación depende del commit, push y despliegue correspondiente.
Público de la aplicación	Estudiantes de colegio, especialmente de quinto de secundaria, y administradores autorizados.
Enfoque del reporte	Descripción cualitativa de siete indicadores respaldada por fragmentos literales de la conversación.

La guía usa ejemplos didácticos y nombres de archivos. No incluye contraseñas, conversaciones de estudiantes ni registros personales.

Contenido

Puedes leer los capítulos 1 a 6 para entender el producto y sus resultados. Los capítulos 7 a 13 explican el código y la operación. El cierre reúne las comprobaciones y un glosario.

01 Qué hace el sistema 3

02 Recorrido de una conversación 4

03 Dimensiones e indicadores 5

04 Cómo se construye el análisis 6

05 Qué significa cada estado 7

06 El Excel y el perfil del alumno 8

07 Frontend: organización y archivos 9

08 Backend: organización y archivos 13

09 Base de datos y relaciones 18

10 Modelos y rendimiento 19

11 Autenticación y conservación 20

12 Cómo se conservan los 110 casos 21

13 Ejecución local y despliegue 22

14 Pruebas y lectura responsable 24

15 Glosario y mapa de lectura 25

01 Qué hace el sistema

ChatBotCognitivo ofrece una conversación personal y continua. El estudiante escribe con libertad sobre su experiencia en el colegio. El asistente responde de manera breve, cercana y contextual; no presenta un formulario de respuestas para marcar.

El sistema tiene dos trabajos diferentes: acompañar la conversación y organizar información expresada por el estudiante. El primero produce una respuesta; el segundo identifica citas relacionadas con las dimensiones e indicadores del proyecto. Separarlos permite que el estudiante no espere a que termine todo el análisis para recibir una respuesta.

Quién utiliza cada parte

Rol	Acciones principales
Estudiante	Crear una cuenta por usuario y contraseña, aceptar la política, iniciar sesión, conversar y continuar su historial.
Administrador	Consultar los estudiantes del estudio, abrir su análisis, leer evidencias y descargar el Excel autorizado.
Worker	Proceso automático que analiza mensajes pendientes, mantiene reportes aplicables y ejecuta la limpieza de mensajes vencidos.

Las tres piezas principales

Frontend: React y TypeScript presentan las pantallas y las interacciones. **Backend:** Django comprueba permisos, gestiona los registros y entrega la API. **Ollama:** ejecuta los modelos de lenguaje que generan respuestas y extraen evidencias.

La base configurada actualmente es **SQLite**. El proyecto no utiliza PostgreSQL en esta versión. En el VPS, la base está dentro del volumen persistente de datos; en desarrollo, se usa la base local configurada en Django.

Qué cambia con la versión cualitativa

El perfil y el Excel pasan de mostrar una frecuencia no precisada a presentar un estado, una explicación y sus evidencias. Las siete columnas de análisis siempre contienen texto. Se distinguen el tema no mencionado, el análisis aún pendiente y la ausencia expresada por la propia persona.

02 Recorrido de una conversación

01 El estudiante escribe

ChatbotPage organiza la vista; useChatbot administra el envío y el estado.

↓

02 La API recibe el mensaje

Django valida la sesión, el rol, el contenido y la clave de envío.

↓

03 Se guarda y se responde

El backend conserva el mensaje y solicita la respuesta al modelo de chat.

↓

04 La respuesta llega por partes

El navegador recibe fragmentos NDJSON y actualiza la burbuja del asistente.

↓

05 El worker analiza

DeepSeek extrae citas; el servidor comprueba formato y pertenencia literal.

↓

06 El administrador consulta

Los indicadores y el Excel se calculan a partir de la evidencia disponible.

El análisis posterior no debe bloquear la respuesta del chat. Si hay una respuesta activa, el worker cede prioridad y vuelve a intentar su trabajo después. Los estados de reserva ayudan a evitar que dos procesos analicen o respondan el mismo mensaje de manera simultánea.

Por qué el historial no se borra al limitar el contexto

El modelo de conversación recibe una ventana reciente del historial, con límite de mensajes y presupuesto de contexto. La base conserva el historial completo mientras su política de conservación lo permita. Reducir lo enviado al modelo es una medida de rendimiento; no es una eliminación de registros.

03 Dimensiones e indicadores

El sistema utiliza la agrupación de tres dimensiones y siete indicadores proporcionada para el proyecto. Son categorías que ayudan a organizar lo que aparece en el relato. Una categoría cubierta significa que existe evidencia; no representa por sí sola una intensidad de estrés.

Dimensión	Indicadores y significado
Estresores académicos	Sobrecarga académica: tareas, exámenes o trabajos que la persona describe como difíciles de asumir. Presión del tiempo: plazos, prisas o falta de tiempo para sus actividades escolares.
Síntomas	Síntomas físicos: molestias corporales, cansancio o sueño que la persona vincula con sus estudios. Síntomas psicológicos: nervios, preocupación, irritación, tristeza o concentración en contexto académico.
Estrategias de afrontamiento	Planificación: organización y prioridades de estudio. Automotivación: prácticas propias para sostener el esfuerzo. Búsqueda de soporte: pedir o recibir apoyo de otras personas.

Una dificultad y un recurso no significan lo mismo

En los cuatro primeros indicadores se describen factores o malestares expresados. En afrontamiento puede observarse que el estudiante utiliza un recurso, que tiene dificultades para utilizarlo o que declara que no lo usa. Emplear planificación o pedir ayuda no equivale a presentar más estrés.

Relación con la tesis

La matriz de dimensiones orienta el prompt de extracción, la estructura de evidencias, los apartados administrativos y las columnas de exportación. Así se conserva la relación entre la conversación y la categoría que se analiza. Esta implementación no reproduce una aplicación completa del inventario SISCO ni convierte automáticamente el relato libre en una puntuación validada.

Para una investigación, las categorías permiten revisar patrones, contrastar citas y describir recursos o dificultades. Una comparación PRE-POST o una afirmación de eficacia requiere un diseño, momentos de medición e instrumentos apropiados; no se obtiene únicamente por disponer de conversaciones.

04 Cómo se construye el análisis

1. Extraer una cita del mensaje del estudiante

El modelo de análisis recibe el mensaje actual y contexto acotado. Debe devolver un JSON con los indicadores y las señales permitidas. Una cita tiene que copiarse del mensaje actual: lo que dijo antes el asistente no demuestra que el estudiante tenga ese problema ni que use ese recurso.

2. Validar antes de guardar

El backend comprueba los códigos de indicador y señal, el tipo de dato y que el fragmento aparezca literalmente en el contenido del mensaje. Los textos inventados, vacíos o ajenos al mensaje se descartan. También se conserva la negación, porque cambiarla alteraría la interpretación.

3. Reunir solo la actividad pertinente

Al preparar el perfil se comprueba la pertenencia del mensaje a su usuario y sesión, su fecha y su estado de análisis. Para los miembros de la muestra, el análisis del estudio se delimita a la ventana registrada del 11 de septiembre de 2026, de 13:00 a 18:00, hora de Lima. El historial personal sigue conservando sus registros.

4. Resumir la evidencia disponible

La capa cualitativa reúne las señales de cada indicador y las observaciones validadas. Produce un estado, una etiqueta, una explicación y citas de respaldo. La exportación utiliza esa misma fuente; no hace otra llamada al modelo ni elige un resultado diferente del perfil.

5. Conservar límites y matices

Si hay señales distintas, se muestra un relato mixto; no se promedian ni se elimina la evidencia menos conveniente. Si quedan mensajes por analizar, el resultado señala su carácter parcial. Ningún número de mensajes o citas se convierte en gravedad de estrés.

La validación literal comprueba que la cita existe. La clasificación del modelo continúa siendo una interpretación susceptible de error y puede requerir revisión humana. Por ello, el reporte conserva el texto que permite comprobar de dónde sale cada observación.

05 Qué significa cada estado

Estado visible	Cuándo se utiliza y cómo leerlo
Dificultad reportada	Hay evidencia válida de una dificultad relacionada con el indicador. Describe lo expresado; no atribuye un diagnóstico.
Recurso utilizado	La persona relata que utiliza una estrategia, por ejemplo organizar sus tareas o pedir ayuda. Una sugerencia del asistente no basta.
Ausencia expresada	La persona niega de forma explícita una dificultad o declara ausencia de un recurso. En afrontamiento se aclara que no usar un recurso no significa ausencia de estrés.
Relato mixto	Se registran señales distintas en el indicador. Pueden reflejar cambios, contextos o contradicciones; se mantienen las citas y se describen los matices.
No mencionado en la conversación	Los mensajes analizados no aportan evidencia válida del indicador. No permite confirmar que la persona carezca de ese problema.
Análisis pendiente	Todavía hay mensajes que requieren análisis y no existe evidencia suficiente para ese indicador. El resultado puede ampliarse al terminar el procesamiento.
Sin conversación registrada	No hay mensajes del estudiante disponibles dentro del periodo consultado. Se informa este estado en lugar de dejar una celda vacía.

Tres ejemplos de lectura

“No me duele la cabeza por los estudios”: si el sistema valida y clasifica ese fragmento, puede describir una ausencia expresada de ese malestar. No autoriza a descartar todos los demás síntomas físicos.

“Hago una lista para ordenar mis tareas”: puede respaldar planificación como recurso utilizado. **“Hoy fui al colegio”:** no aporta por sí solo evidencia de planificación ni de síntomas. Se registra el tema como no mencionado, sin asumir una ausencia.

Estos ejemplos son didácticos. No proceden de los registros de una persona.

06 El Excel y el perfil del alumno

Hoja Resultados: una fila por estudiante

La descarga general del estudio incluye a los 110 seleccionados. La descarga desde un detalle puede incluir al estudiante consultado, siempre que pertenezca a la muestra autorizada. El servidor valida los IDs; cambiar la petición desde el navegador no permite añadir a los otros 26.

Columnas	Contenido
A - ID estudiante	Identificador estable del registro, presentado como Estudiante seguido de su ID.
B - Usuario	Nombre de usuario con el que se identifica la cuenta.
C a I - Siete indicadores	Estado y análisis cualitativo por sobrecarga académica, presión del tiempo, síntomas físicos, síntomas psicológicos, planificación, automotivación y búsqueda de soporte.

Las siete celdas de análisis contienen texto. Cuando hay evidencia, describen lo relatado y su respaldo. Cuando no la hay, indican el estado correspondiente. La versión cualitativa no utiliza una celda numérica para representar un problema que no se ha mencionado.

Hoja Evidencias: trazabilidad

Reúne las citas validadas con identificación del estudiante, usuario, dimensión, indicador, estado, observación y fecha del mensaje en hora de Lima. Esta hoja permite revisar los fragmentos detrás del resumen. Un indicador sin evidencia no genera una cita inventada para completar la tabla.

Hoja Ficha: método e interpretación

Explica la fecha de exportación, el periodo, la selección registrada, el significado de las categorías y el alcance de la lectura. Cuando un registro conserva una procedencia de demostración, el archivo mantiene esa información. Cambiar el formato de exportación no modifica la procedencia del dato.

Perfil cualitativo en la pantalla

La pantalla organiza los siete indicadores en tres grupos. Cada tarjeta muestra su estado y explicación; las citas se abren con un desplegable y muestran su fecha. El perfil puede ofrecer una selección breve de evidencias, mientras la descarga conserva el detalle exportable. La síntesis final reúne las dificultades y los recursos expresados.

07 Frontend: organización y archivos

La carpeta frontend contiene la aplicación que se ejecuta en el navegador. La combinación React + TypeScript permite dividir la interfaz en piezas y definir la forma de los datos. Vite sirve el proyecto en desarrollo y genera la carpeta dist para producción.

Ruta en frontend/	Responsabilidad
src/pages/	Pantallas completas: acceso, registro, chat y administración. Componen elementos y llaman a hooks; concentran lo necesario para cada ruta.
src/components/	Piezas específicas del producto: conversación, barra lateral, tarjetas y reportes. Se reutilizan en las pantallas.
src/hooks/	Estado y coordinación de la aplicación: usuario autenticado, mensajes, consultas y actualización de datos administrativos.
src/api/	Peticiones HTTP y contratos de comunicación con Django. Separa la red del código de presentación.
src/routes/	Rutas de React Router, accesos por rol y redirecciones.
src/types/	Interfaces TypeScript para usuarios, chat, alumnos y reportes.
src/ui/	Controles reutilizables: botón, entrada de texto y botón de icono.
src/utils/	Funciones auxiliares, como el formato de fechas.
src/assets/ y public/	Recursos estáticos del frontend. Su presencia no implica que todos se estén usando en una pantalla.
dist/ y node_modules/	Salida de compilación y dependencias instaladas. Se generan; no son el código fuente principal.

Entrada y navegación

Archivo	Qué hace
main.tsx	Monta React en el elemento root e incorpora los estilos y el proveedor de navegación.
App.tsx	Compone la aplicación con su proveedor de autenticación y el sistema de rutas.
index.css	Define Tailwind, fuentes, colores y reglas generales para ancho, entradas y comportamiento visual.
routes/AppRoutes.tsx	Define /login, /registro, /privacidad, /chat y /administrador. Comprueba el rol antes de mostrar rutas privadas y conserva redirecciones antiguas.

Pantallas del estudiante

Archivo en src/pages/	Qué hace
`LoginPage.tsx`	Recoge usuario y contraseña y solicita iniciar sesión. El identificador es texto, con autocompletado de usuario.
`RegistroPage.tsx`	Solicita usuario, contraseña y confirmación. Valida el formato y exige la aceptación explícita de la política vigente.
`PrivacidadPage.tsx`	Consulta y presenta las condiciones de uso y conservación entregadas por el servidor.
`ChatbotPage.tsx`	Compone barra lateral, cabecera, historial, respuesta parcial, estado de escritura y entrada de mensajes.

Pantallas administrativas

Archivo en pages/admin/	Qué hace
`LoginAdminPage.tsx`	Presenta el acceso administrativo. La verificación final del permiso siempre corresponde al backend.
`AdminPage.tsx`	Muestra población y casos seleccionados, dimensiones, listado y botón de Excel para los integrantes de la muestra.
`AlumnoPage.tsx`	Presenta actividad, análisis actual, reportes conservados y perfil cualitativo del alumno.
`ReporteAlumnoPage.tsx`	Abre un reporte guardado y sus dimensiones. Incluye el perfil cualitativo disponible del alumno.

Hooks: mantener las pantallas legibles

Archivo en src/hooks/	Qué hace
`useAuth.ts`	Expone el contexto de autenticación para iniciar, registrar y cerrar la sesión.
`useChatbot.ts`	Recupera el historial, administra el texto, el envío, la respuesta parcial, errores, reintentos y referencias de desplazamiento.
`useDatosAdmin.ts`	Coordina la carga de datos administrativos, sus errores y la actualización periódica.
`useAlumnos.ts`	Entrega el listado y los metadatos del estudio a la pantalla administrativa.
`useAlumno.ts`	Consulta el detalle de un estudiante por ID.
`useReportes.ts`	Consulta el detalle de un reporte por ID.

API del frontend

Archivo en src/api/	Qué hace
`http.ts`	Centraliza fetch, credenciales de sesión, CSRF, interpretación de errores y comprobación de respuestas.
`authApi.ts`	Registro, acceso, sesión actual, cierre y consulta de términos. Normaliza el usuario a minúsculas.
`chatbotApi.ts`	Historial y envío del chat. Lee los eventos NDJSON, conserva fragmentos y controla errores de flujo.
`alumnosApi.ts`	Listado del estudio y detalle administrativo de cada alumno.
`reportesApi.ts`	Consulta de reportes guardados.
`exportacionApi.ts`	Solicita y valida el archivo Excel, con sesión y selección de hasta 110 alumnos.

Componentes y controles

Archivo o grupo	Qué hace
`BarraLateralComp.tsx`	Identificación de la cuenta, acceso al chat, política aplicable y cierre de sesión.
`CabeceraChatComp.tsx`	Cabecera de la conversación y apertura del menú móvil.
`EntradaChatComp.tsx`	Campo del mensaje, botón de envío y combinaciones Enter / Shift+Enter.
`MensajeChatComp.tsx`	Representación de un mensaje según remitente y contenido.
`InicioChatComp.tsx`	Presentación inicial cuando aún no hay conversación.
`EstadoEscribiendoComp.tsx`	Indicador visual mientras se espera el primer fragmento de respuesta.
`auth/AuthProviderComp.tsx`	Mantiene la identidad autenticada y las operaciones de acceso compartidas.
`auth/MarcoAuthComp.tsx`	Estructura visual común del registro, acceso y política.
`admin/MarcoAdminComp.tsx`	Cabecera, navegación, contenedor y cierre de sesión del administrador.
`admin/TablaAlumnosComp.tsx`	Tabla en escritorio y tarjetas en móvil para abrir un estudiante.
`admin/InformacionAlumnoComp.tsx`	Fechas y recuentos de actividad del alumno. No muestra los dos campos retirados de última actividad y tiempo transcurrido.
`admin/ReporteActualComp.tsx`	Cobertura y dimensiones del análisis disponible al consultar.
`admin/DimensionReporteComp.tsx`	Presenta indicadores y evidencias de una dimensión.
`admin/PerfilIndicadoresComp.tsx`	Tarjetas cualitativas por indicador, estado, análisis y citas desplegables.
`admin/TarjetaReporteComp.tsx`	Resumen enlazado de un reporte guardado.
`admin/DescargarEvaluacionComp.tsx`	Botón, estado de descarga y errores del Excel.
`admin/EstadoCargaComp.tsx`	Mensajes de carga, fallo y reintento de consultas.
`ui/ y utils/fechas.ts`	Controles visuales básicos y formato de fechas en hora de Lima.

08 Backend: organización y archivos

La carpeta backend contiene el entorno Python, el proyecto Django y los archivos de ejecución en producción. Dentro de backend/principal se encuentra manage.py, las aplicaciones y la carpeta de configuración principal. La carpeta ml es un módulo normal de Python; no es una aplicación Django instalada con sus propias tablas.

Configuración y puntos de entrada

Archivo	Qué hace
`manage.py`	Ejecuta comandos de Django: migraciones, comprobaciones, pruebas, arranque y tareas administrativas.
`principal/settings.py`	Configura aplicaciones, base SQLite, seguridad, cookies, modelo de chat, modelo de análisis y exigencia de muestra en producción. Lee variables del entorno.
`principal/urls.py`	Asocia rutas HTTP con vistas del chat, autenticación, alumnos, reportes y Excel.
`principal/http.py`	Respuestas de error, lectura de JSON, comprobaciones de método, autenticación y rol. También expone salud y fallo CSRF.
`principal/wsgi.py`	Entrada de la aplicación para Gunicorn en producción.
`principal/asgi.py`	Entrada ASGI disponible del proyecto. Su existencia no implica que el despliegue actual utilice un servidor ASGI.

Aplicación usuarios

Archivo o comando	Qué hace
`usuarios/models.py`	Define Perfil: actividad, procedencia, aceptación de términos y reserva de respuesta del usuario.
`usuarios/views.py`	CSRF, términos, sesión actual, registro, login y logout. Valida el username y las contraseñas, y renueva la retención aplicable al iniciar sesión.
`usuarios/retencion.py`	Versiona políticas, asigna vencimientos, renueva la versión de 90 días y elimina lotes elegibles con sus derivados.
`preparar_admin_produccion`	Prepara el administrador del despliegue con la configuración recibida, sin cambiar credenciales innecesariamente.
`crear_admin / preparar_admin`	Herramientas administrativas de creación o preparación de una cuenta administradora en los flujos correspondientes.
`limpiar_conversaciones`	Consulta o aplica el borrado de mensajes elegibles. No equivale a eliminar las cuentas.
`carga_actividad.py / cargar_actividad`	Validación y carga controlada de lotes de actividad. Son herramientas de importación, no se ejecutan automáticamente con cada mensaje.
`cambio_usuarios.py`	Herramienta de conversión de identidades según un mapa controlado, conservando relaciones y contraseñas cuando coincide la identidad esperada.
`management/commands/runserver.py`	Integra la preparación local prevista por el proyecto al ejecutar el servidor de desarrollo.

Aplicación chat

Archivo o comando	Qué hace
`chat/models.py`	Define Mensaje, su respuesta asociada, fecha, clave de envío, estado de análisis y vencimiento.
`chat/views.py`	Recibe peticiones del historial y del envío, y devuelve la respuesta normal o el flujo NDJSON.
`chat/services.py`	Coordina guardado, contexto reciente, reserva de respuesta, idempotencia, llamada al modelo y conservación del historial.
`precalentar_modelo`	Solicita a Ollama que prepare los modelos para reducir el coste de una primera carga.

Módulo ml

Archivo	Qué hace
`ml/conversacion.py`	Reglas del acompañamiento escolar, respuestas breves a saludos aislados, construcción de respuesta y normalización del contexto escolar.
`ml/analisis.py`	Dimensiones, indicadores, señales, esquema JSON, instrucciones y ejemplos de extracción de citas.
`ml/ollama.py`	Cliente HTTP de Ollama, presupuesto de contexto, modelos, límites, lectura del flujo, cancelación y precalentamiento.
`ml/__init__.py`	Identifica el directorio como módulo importable de Python.

Aplicación sesiones

Archivo o comando	Qué hace
`sesiones/models.py`	Define el contenedor temporal del historial. Su nombre histórico es SesionSemanal.
`sesiones/services.py`	Recupera o crea la conversación abierta y conserva los periodos históricos.
`iniciar_backend`	Orquesta el inicio local del backend según el flujo del proyecto.
`procesar_semanas`	Nombre histórico del worker. En modo continuo procesa pendientes, reportes aplicables y limpieza; no impone cuatro semanas al chat actual.

Aplicación reportes

Archivo o comando	Qué hace
`reportes/models.py`	Define evidencias, reportes guardados y la muestra persistente del estudio.
`reportes/services.py`	Reserva análisis, valida la extracción, conserva evidencia y prepara cobertura, dimensiones y análisis actual.
`reportes/indicadores.py`	Reúne mensajes/evidencias admisibles y construye estados, explicación y respaldo del perfil cualitativo compartido.
`reportes/exportacion.py`	Construye Resultados, Evidencias y Ficha usando la misma interpretación cualitativa; entrega un XLSX en memoria.
`reportes/frecuencias.py`	Lógica conservada de frecuencias explícitas para compatibilidad técnica. La nueva presentación cualitativa no depende de asignar una frecuencia a cada alumno.
`reportes/estudio.py`	Localiza y valida la muestra, entrega sus metadatos y delimita la actividad de sus integrantes a la ventana del estudio.
`reportes/views.py`	Listado administrativo, detalle de alumno, reporte y descarga. Comprueba permisos y selección autorizada.
`seleccionar_estudio`	Crea una selección de 110 solo con la población elegible esperada; en reinicios conserva los mismos IDs.

Archivos generados y pruebas

Las carpetas migrations contienen cambios versionados del esquema. Los archivos tests.py y test_*.py comprueban reglas y recorridos. __pycache__ y .venv son generados localmente; .idea es configuración de un IDE. No deben confundirse con las aplicaciones ni incorporarse como datos del estudio.

Análisis actual y reporte guardado

La conversación abierta obtiene su análisis actual al consultar reporte_conversacion_json(). El worker actualiza reportes persistidos de sesiones cerradas cuando corresponde. Por tanto, una conversación abierta puede ofrecer un análisis administrativo aunque no exista un nuevo ReporteSemanal guardado. Los nombres históricos no significan un cierre automático cada siete días.

Rutas principales de la API

Método y ruta	Propósito
`GET /api/salud/`	Comprueba servidor y conexión a base de datos.
`GET /api/csrf/`	Entrega el token de seguridad para escrituras.
`GET /api/auth/terminos/`	Entrega la política vigente y su versión.
`POST /api/auth/registro/`	Crea una cuenta sin privilegios con aceptación explícita.
`POST /api/auth/login/`	Autentica usuario y contraseña; renueva el plazo de la política aplicable.
`POST /api/auth/logout/`	Cierra el acceso autenticado.
`GET /api/auth/me/`	Consulta identidad y actividad; no renueva el plazo de retención.
`GET /api/chat/`	Recupera el historial de la persona autenticada.
`POST /api/chat/flujo/`	Envía un mensaje y recibe la respuesta progresiva.
`GET /api/admin/alumnos/`	Listado de los integrantes del estudio autorizado.
`GET /api/admin/alumnos/<id>/`	Detalle del estudiante, análisis actual y perfil.
`GET /api/admin/reportes/<id>/`	Detalle de un reporte guardado.
`GET /api/admin/evaluacion.xlsx`	Descarga protegida con los IDs solicitados en el parámetro alumnos.

09 Base de datos y relaciones

Django transforma los modelos en tablas mediante migraciones. Las claves foráneas mantienen la relación entre identidad, conversación, mensaje y evidencia. Conservar los IDs evita separar un mensaje de su cuenta al cambiar el nombre de usuario.

Tabla	Qué guarda y cómo se relaciona
`auth_user`	Cuenta Django: ID, username, contraseña codificada mediante hash, nombres internos, email heredado, is_staff, is_superuser, is_active y last_login. El registro nuevo no solicita correo ni nombre personal.
`usuarios_perfil`	Una fila por usuario: inicio, actividad, conectado, reserva de respuesta, procedencia y fecha/versión de términos. usuario_id es único.
`sesiones_sesionsemanal`	Periodos de conversación por usuario: número, inicio, fin y estado. fin puede ser NULL para una conversación abierta. usuario+número es único.
`chat_mensaje`	Contenido y remitente, fecha, usuario_id, sesion_id, clave de envío, respuesta asociada, errores, estado e intentos de análisis, vencimiento y versión de retención.
`reportes_evidencia`	Fragmento literal, observación, indicador y señal. mensaje_id identifica la fuente. Se evita duplicar la misma cita del mismo indicador en un mensaje.
`reportes_reportesemanal`	Resumen guardado, estado, fechas y recuentos. sesion_id es único. El nombre se conserva por compatibilidad.
`reportes_muestraestudio`	Clave del estudio, inicio/fin, tamaño, lista original de IDs de población, método y fecha de selección.
`reportes_muestraestudio_usuarios`	Tabla intermedia de muchos a muchos. Guarda qué 110 usuarios pertenecen a la muestra, sin duplicar sus cuentas ni sus mensajes.

Relaciones explicadas

```
Usuario 1:1 Perfil; Usuario 1:N Sesiones y Mensajes
Sesion 1:N Mensajes; Sesion 1:0..1 Reporte guardado
Mensaje 1:N Evidencias; Mensaje 1:0..1 Respuesta
Muestra N:M Usuarios
```

Un mensaje tiene usuario y sesión; el servidor comprueba que ambos correspondan a la misma persona. Una evidencia no crea un estudiante nuevo: apunta al mensaje fuente. Una muestra añade relaciones a cuentas existentes; no copia datos ni altera sus contraseñas.

Infraestructura Django: django_migrations conserva migraciones; django_session, sesiones de acceso; auth_group y auth_permission, permisos; django_content_type, modelos; django_admin_log, operaciones administrativas. No son indicadores de estrés.

Borrado y persistencia: CASCADE elimina dependencias, como evidencias de un mensaje borrado. La retención retira resúmenes derivados y conserva la cuenta. El volumen Docker mantiene la base al reemplazar un contenedor.

10 Modelos y rendimiento

Modelo de conversación

OLLAMA_CHAT_MODEL configura el modelo que responde al estudiante; el valor por defecto del proyecto es **qwen3:1.7b**. El prompt pide español sencillo, respuestas de una a tres frases y como máximo una pregunta natural cuando sea útil. El contexto se mantiene escolar y no se interroga para completar indicadores.

Modelo de análisis

OLLAMA_MODEL configura el modelo de extracción; por defecto es **deepseek-r1:8b**. Recibe reglas y un esquema que limita las salidas a indicadores y señales permitidas. Extrae evidencia; no responde al estudiante durante esa tarea.

Qué entrenamiento existe

El repositorio no implementa ajuste fino de los pesos del modelo ni un entrenamiento supervisado propio. Utiliza modelos preentrenados ejecutados por Ollama. La adaptación se realiza con instrucciones, ejemplos de formato, contexto reciente y validación de la salida. Añadir un ejemplo al prompt no equivale a reentrenar el modelo.

Medidas de rapidez

Medida	Qué aporta
Modelo de chat más ligero	Reduce el trabajo necesario para producir una respuesta respecto a usar el modelo mayor para todo.
Respuesta por fragmentos	Muestra contenido mientras se genera; la percepción de espera no depende de esperar la respuesta completa.
Contexto reciente acotado	Evita reenviar el historial entero al modelo en cada intervención.
Precalentamiento y keep_alive	Intentan conservar el modelo preparado entre peticiones.
Prioridad del chat	El worker evita competir con una respuesta activa y reintenta tareas pendientes.

El tiempo depende de CPU/GPU, memoria, tamaño del contexto, carga y modelo. Esta guía no garantiza una latencia fija. El modelo pequeño tiene menor capacidad y también puede equivocarse; el control del formato y la revisión de evidencia siguen siendo necesarios.

11 Autenticación y conservación

Usuario y contraseña

El username tiene entre 3 y 40 caracteres, empieza con letra y admite letras ASCII, números, punto, guion y guion bajo. Se normaliza a minúsculas. El servidor rechaza duplicados sin distinguir mayúsculas y crea las cuentas nuevas sin privilegios. Mantiene compatibilidad para identificadores de cuentas anteriores.

La contraseña se valida con las reglas Django y se almacena con hash, no como texto legible. Las peticiones usan autenticación de sesión. Una cookie identifica la sesión; el backend vuelve a comprobar el rol en las rutas administrativas. Ocultar un botón en React no sustituye esa comprobación.

CSRF y transporte

Las operaciones de escritura incluyen un token CSRF para impedir peticiones no autorizadas desde otros sitios. En producción se configuran HTTPS, orígenes de confianza y cookies seguras. Las credenciales del administrador y la clave secreta se leen del entorno y no forman parte del PDF ni del Excel.

Aceptación explícita

El registro presenta una casilla desmarcada y requiere aceptar la política vigente. Django guarda su versión y la fecha de aceptación. No se atribuye a una cuenta histórica una aceptación que no realizó.

Plazo de 90 días desde el inicio de sesión

Para la nueva política, los mensajes elegibles vencen a los 90 días del último inicio de sesión correcto. Un login correcto renueva el plazo de los mensajes sujetos a esa versión. La consulta periódica de sesión, un latido o un acceso fallido no lo renuevan.

Ejemplo de funcionamiento: si una cuenta con la nueva política inicia sesión el 1 de octubre y vuelve a iniciar sesión el 20 de octubre, el plazo se calcula desde el 20 de octubre. La aplicación consulta las fechas almacenadas; el borrado se aplica en el ciclo posterior de limpieza que corresponda.

Qué se mantiene y qué se elimina

Elemento	Tratamiento
Mensajes elegibles vencidos	El worker elimina los lotes que cumplen las condiciones de la política, evitando respuestas activas.
Citas y resúmenes derivados	Se retiran para no retener el contenido eliminado a través de los reportes.
Cuenta del usuario	Permanece; el plazo de los mensajes no es un plazo de eliminación de la cuenta.
Versión anterior de 30 días	Conserva su regla previamente aceptada, sin reescritura automática a la versión nueva.
Datos históricos sin vencimiento	No se inscriben ni eliminan retroactivamente por introducir la política nueva.
Excel descargado y respaldos	Tienen una gestión de conservación independiente; la aplicación no borra copias externas ya descargadas.

12 Cómo se conservan los 110 casos

La selección se realiza sobre las 136 cuentas elegibles con mensajes del estudiante en la ventana del 11 de septiembre de 2026, de 13:00 a 18:00, America/Lima. Se eligen 110 sin reemplazo y se guardan sus IDs en la base. El criterio de selección no utiliza niveles de estrés ni resultados favorables.

Primer despliegue y posteriores reinicios

Una migración inicial puede preparar la selección cuando ya existe la población exacta. Además, el arranque del backend ejecuta seleccionar_estudio con --aplicar y --si-procede. Si la muestra ya existe y es válida, la reutiliza. Si los datos llegaron después de la migración, el comando cubre ese caso cuando arranca el servicio.

```
python manage.py seleccionar_estudio --aplicar --si-procede
```

Si todavía no están las 136 cuentas elegibles, el comando deja la selección pendiente. Si una muestra existente está incompleta o contiene cuentas ajenas, no la repone ni la vuelve a sortear. Esa situación requiere revisar el dato que falta, manteniendo visible el problema.

Listado, descarga y conservación

En producción, el listado del estudio y su descarga exigen una muestra válida de 110. Si falta, devuelven un aviso de indisponibilidad en vez de incluir silenciosamente a toda la población. Una cuenta nueva no se añade a una selección ya guardada.

Los otros 26 estudiantes permanecen en la base con sus relaciones. Se excluyen del listado y la exportación del estudio. El administrador puede conservar el total de población 136 como contexto, mientras trabaja con 110 casos seleccionados.

El código y los datos viajan de forma diferente

El commit y el push publican código al repositorio. El despliegue reemplaza contenedores y ejecuta migraciones, mientras el volumen conserva la base del VPS. El flujo preparado no reemplaza esa base por la SQLite del equipo local ni vuelve a importar los lotes de actividad anteriores.

13 Ejecución local y despliegue

Inicio local en Windows

iniciar.bat llama a iniciar.ps1. El script comprueba herramientas, prepara el entorno Python, instala dependencias, verifica Ollama y los modelos, aplica migraciones e inicia backend y frontend. La primera preparación puede necesitar Internet, espacio en disco y tiempo de descarga. El script no elimina los datos del VPS.

Ruta o archivo	Uso
`iniciar.bat`	Entrada de doble clic en Windows. Muestra instrucciones y conserva el error si el inicio falla.
`iniciar.ps1`	Coordina instalación local, servicios, salud, logs y apertura del navegador.
`herramientas/`	Ayudantes de preparación e importación. Conviene revisar cada herramienta antes de ejecutar una carga.
`backend/requirements.txt`	Dependencias Python de la aplicación.
`frontend/package.json`	Dependencias y comandos npm de desarrollo, pruebas y compilación.
`frontend/vite.config.ts`	Servidor de desarrollo y configuración de Vite, incluido el acceso a la API local.

Direcciones habituales del inicio local: frontend en http://localhost:5173 y API Django en 127.0.0.1:8000. La ruta administrativa del frontend es /administrador. Ollama normalmente escucha en 127.0.0.1:11434. Estas direcciones locales no son el dominio público.

Servicios Docker Compose

Servicio	Trabajo
frontend	Compila React y sirve la aplicación mediante Nginx. Envía las rutas de API al backend.
backend	Gunicorn sirve Django. El arranque aplica migraciones, comprueba la muestra, prepara el administrador y los estáticos y precalienta el modelo.
worker	Utiliza el código del backend y ejecuta procesar_semanas en modo continuo. Comparte el volumen de datos y espera la salud del backend.
ollama	Servicio de modelos cuando se activa el perfil previsto. Conserva modelos en su volumen.
modelo	Comprueba disponibilidad de Ollama y prepara los modelos configurados antes de iniciar los servicios dependientes.

Archivos del despliegue

Archivo	Qué prepara
`docker-compose.yml`	Servicios, dependencias, red privada, healthchecks, variables y volúmenes.
`backend/Dockerfile`	Imagen Python con dependencias y ejecución bajo un usuario de aplicación sin privilegios de root.
`backend/docker/entrypoint.sh`	Preparación idempotente antes de Gunicorn. El worker no vuelve a aplicar migraciones en su arranque.
`backend/gunicorn.conf.py`	Configuración del servidor WSGI en producción.
`frontend/Dockerfile`	Compilación del frontend y construcción de la imagen de Nginx.
`frontend/nginx.conf`	Entrega del frontend, rutas de API y comportamiento del proxy y del flujo.
`preparar_despliegue.py`	Ayuda a preparar la configuración de despliegue. Las claves generadas deben mantenerse privadas.
`.gitignore / .gitattributes`	Exclusiones de secretos/generados y reglas de versionado/formato, respectivamente.

Publicar una modificación

Primero se revisan y prueban los cambios locales. Después se realiza commit y push a la rama configurada. Dokploy obtiene el código y construye/despliega los servicios según su configuración. Un commit solo en el equipo no publica la web. Tras el despliegue deben comprobarse salud, acceso, selección y descarga.

Los volúmenes datos, estaticos y modelos cumplen funciones distintas. datos conserva /data/db.sqlite3; estaticos comunica los archivos estáticos de Django; modelos evita descargar de nuevo los pesos de Ollama. No se deben sustituir por una base local al actualizar el código.

14 Pruebas y lectura responsable

Qué se verifica en el proyecto

Área	Comprobaciones relevantes
Identidad y permisos	Formato y duplicados del usuario, términos, contraseña, rol del administrador, CSRF y acceso por sesión.
Chat	Historial propio, orden de mensajes, idempotencia en reintentos, fragmentos UTF-8, cancelación y errores de red/modelo.
Evidencia	Citas literales, propietario, sesión, fecha, señal permitida, negación y mensajes pendientes.
Cualitativo	Distinción entre tema no mencionado, ausencia expresada, relato mixto y pendiente. Celdas con texto y respaldo cuando existe.
Muestra	110 IDs persistidos, otros 26 conservados, reinicios sin sorteo nuevo y rechazo de cuentas ajenas en el Excel.
Retención	Vencimiento desde login correcto, renovación aplicable, protección de datos históricos y conservación de la cuenta.
Producción	Configuración Compose, migraciones y reglas que impiden mostrar todos los usuarios si falta la muestra exigida.

Comandos de referencia

```
cd backend/principal
python manage.py check
python manage.py test --noinput
python manage.py makemigrations --check --dry-run

cd frontend
npm run test:api
npm run build
npm run lint
```

Límites del resultado

La aplicación describe el contenido expresado y clasificado. No certifica que una persona tenga o no tenga un trastorno, no reemplaza una entrevista profesional y no demuestra que la intervención reduzca el estrés. La cantidad de mensajes mide actividad; la cantidad de indicadores con citas mide cobertura temática.

El relato puede omitir asuntos, cambiar con el tiempo o contener matices. Una clasificación cualitativa debe poder revisarse con la fuente. La procedencia se conserva al importar datos: disponer de un registro en la aplicación no lo convierte automáticamente en una observación empírica válida de investigación.

15 Glosario y mapa de lectura

Término	Significado dentro del proyecto
Frontend	Código de la aplicación visible que se ejecuta en el navegador.
Backend	Servidor que valida, guarda, procesa y entrega datos autorizados.
API	Conjunto de rutas y formatos para que el navegador solicite acciones o datos al servidor.
Endpoint	Una ruta concreta de la API, como /api/chat/.
Componente	Pieza visual React con propiedades y comportamiento delimitados.
Hook	Función React que reúne estado o coordinación reutilizable, por ejemplo useChatbot.
Router	Sistema que decide qué pantalla corresponde a cada URL.
TypeScript / interfaz	Tipos que describen la forma esperada de un dato durante el desarrollo.
JSON	Formato estructurado para intercambiar datos entre servicios.
NDJSON / streaming	Eventos JSON separados por líneas, recibidos de manera progresiva.
Sesión autenticada	Mecanismo con el que Django reconoce a una cuenta que inició sesión.
Sesión de conversación	Contenedor temporal del historial. Es distinto de la sesión de acceso.
Cookie	Dato que el navegador envía al servidor, entre otros usos para mantener una sesión.
CSRF	Protección que exige un token válido en operaciones de escritura desde el navegador.
Hash de contraseña	Resultado no legible que permite verificar una contraseña sin guardarla como texto plano.
Modelo de datos	Clase Django que describe campos y relaciones persistidos en tablas.
Modelo de lenguaje	Sistema de generación/extracción ejecutado por Ollama; no es una tabla de Django.
Migración	Cambio versionado que prepara o modifica el esquema de base de datos.
ID / UUID	Identificador de un registro. Los mensajes usan UUID para distinguir sus entidades.
Clave foránea	Campo que enlaza una fila con una entidad de otra tabla.
Worker	Proceso separado que realiza tareas pendientes sin mantener al usuario esperando.

Término	Significado dentro del proyecto
Prompt	Instrucciones y contexto enviados al modelo para una tarea concreta.
Evidencia / cita	Fragmento literal del mensaje del estudiante asociado a un indicador.
Observación	Interpretación cualitativa que acompaña a una cita y debe poder contrastarse con ella.
Señal	Clasificación básica: dificultad, recurso o ausencia explícita.
Cobertura	Cuántos indicadores o dimensiones cuentan con evidencia. No equivale a gravedad.
Muestra	Conjunto persistente de 110 cuentas elegido entre la población elegible del estudio.
Volumen Docker	Almacenamiento persistente independiente del reemplazo de un contenedor.
Commit / push / despliegue	Guardar una versión local, enviarla al repositorio y ponerla en ejecución en el servidor, respectivamente.

Por dónde comenzar a explicar el código

ChatbotPage.tsx: muestra cómo se arma la conversación. **useChatbot.ts y chatbotApi.ts:** explican el estado y la comunicación. **chat/services.py:** conecta permisos, persistencia y respuesta. **ml/conversacion.py:** define el acompañamiento. **ml/analisis.py y reportes/indicadores.py:** explican la evidencia y su interpretación. **reportes/exportacion.py:** muestra cómo llega el resultado al Excel.

Base utilizada para esta guía

Código fuente del repositorio ChatBotCognitivo, modelos y migraciones Django, contratos TypeScript, configuración de inicio local y Docker Compose, revisados para la versión cualitativa del 29 de octubre de 2026. Las definiciones operativas se refieren a esta implementación y a la matriz de indicadores aportada al proyecto. No se han añadido resultados personales ni citas bibliográficas no verificadas.